

SIMATS ENGINEERING

ASSESSMENT TOOL 2

Shaik. Sameena(192372264)

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4303

Code of Conduct:

I Shaik. Sameena certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools (CSS/JS)	20	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10	Clear, well-structured, and properly	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

		presented solution			
--	--	-----------------------	--	--	--

Question 1: Responsive Navigation Bar Design – 4 Marks

(a) Key Components Required

An e-commerce navigation bar should contain:

- **Logo/Brand name** – identifies the website.
- **Home** – takes the user to the main page.
- **Products/Categories** – allows users to browse products.
- **Search bar** – helps users find products quickly.
- **Cart icon** – displays selected products.
- **Login/Profile** – provides access to the user's account.
- **Navigation menu** – provides easy movement between pages.

(b) Responsive Navigation Bar Using CSS

HTML:

```
<nav class="navbar">
  <div class="logo">ShopEasy</div>

  <div class="nav-links" id="navLinks">
    <a href="#">Home</a>
    <a href="#">Products</a>
    <a href="#">Offers</a>
    <a href="#">Contact</a>
  </div>

  <button class="menu-btn" onclick="toggleMenu()">≡</button>
</nav>
```

CSS:

```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 15px 30px;
```

```
    background: #333;  
}
```

```
.logo {  
    color: white;  
    font-size: 24px;  
    font-weight: bold;  
}
```

```
.nav-links {  
    display: flex;  
    gap: 25px;  
}
```

```
.nav-links a {  
    color: white;  
    text-decoration: none;  
}
```

```
.menu-btn {  
    display: none;  
}
```

```
@media (max-width: 600px) {  
    .nav-links {  
        display: none;  
        flex-direction: column;  
    }  
}
```

```
.nav-links.active {  
    display: flex;  
}
```

```
.menu-btn {
```

```
display: block;
}
}
```

(c) Adaptation to Smaller Screens

On desktop and tablet screens, the navigation links are displayed horizontally. When the screen width becomes smaller than 600px, the links are hidden and a **hamburger button** (≡) is displayed.

JavaScript can display the menu when the hamburger button is clicked:

```
function toggleMenu() {
    document.getElementById("navLinks").classList.toggle("active");
}
```

(d) Usability and Improvements

The navigation should be simple and easy to understand. The hamburger button should be large enough for touch interaction. Improvements include adding a search icon, cart count, clear active-page indicators, keyboard navigation, and accessible labels. The navigation should also be tested on different screen sizes.

Question 2: Product Hover Effect – 4 Marks

(a) Suitable CSS Properties

The following CSS properties are suitable for product hover effects:

- transform
- transition
- box-shadow
- opacity
- background-color
- border
- filter

These properties can make product cards visually interactive.

(b) CSS Animation or Transition

A CSS transition can be used to create a smooth effect when the user moves the mouse over a product.

```
.product-card {
    transition: transform 0.3s ease, box-shadow 0.3s ease;
```

```
}
```

The transform property can slightly enlarge or lift the card.

(c) Product Card Hover Effect

HTML:

```
<div class="product-card">
  
  <h3>Sports Shoe</h3>
  <p>₹1,999</p>
  <button>Buy Now</button>
</div>
```

CSS:

```
.product-card {
  width: 250px;
  padding: 15px;
  border: 1px solid #ddd;
  border-radius: 10px;
  text-align: center;
  transition: transform 0.3s ease,
              box-shadow 0.3s ease;
}

.product-card img {
  width: 100%;
}

.product-card:hover {
  transform: translateY(-8px);
  box-shadow: 0 8px 20px rgba(0,0,0,0.2);
}
```

When the mouse pointer moves over the product card, the card moves slightly upward and displays a shadow.

(d) Performance and User Experience

Hover effects should be smooth and short so that they do not distract users. Lightweight CSS transitions are preferable to complex animations. Excessive animations can reduce

performance. The design should also consider mobile devices because touchscreens do not have traditional hover behavior. Important information should therefore not depend only on hover.

Question 3: Layout Selection – Flexbox vs Grid – 4 Marks

(a) Dashboard Layout Requirements

A dashboard generally contains:

- Header
- Sidebar
- Main content area
- Multiple rows and columns
- Cards for displaying information
- Charts and tables
- Buttons and controls

The layout should provide proper alignment, spacing, and responsiveness.

(b) Comparison of Flexbox and Grid

Feature	Flexbox	CSS Grid
Layout type	One-dimensional	Two-dimensional
Best for	Rows or columns	Rows and columns
Alignment	Excellent	Excellent
Dashboard layouts	Suitable for individual sections	Very suitable
Complex layouts	Less suitable	More suitable
Responsiveness	Good	Excellent

Flexbox is mainly designed for arranging elements along one direction, either a row or a column.

Grid is designed for controlling both rows and columns simultaneously.

(c) Suitable Layout Technique

CSS Grid is the most suitable choice for a dashboard because dashboards normally contain multiple rows and columns.

Example:

```
.dashboard {  
    display: grid;
```

```
grid-template-columns: 200px 1fr 1fr;  
grid-template-rows: 80px 1fr;  
gap: 20px;  
}
```

Grid allows the designer to control the position and size of different dashboard sections easily.

(d) Scalability and Responsiveness

CSS Grid is highly scalable because new cards or sections can be added without redesigning the entire layout.

A media query can make the dashboard responsive:

```
@media (max-width: 768px) {  
  .dashboard {  
    grid-template-columns: 1fr;  
  }  
}
```

On smaller screens, the dashboard sections are placed in a single column. Therefore, Grid provides a flexible and maintainable solution for dashboard applications.

Question 4: Mobile-First Blog Layout – 4 Marks

(a) Key Sections of a Blog Layout

A typical blog contains:

1. Header
2. Navigation menu
3. Blog title
4. Main article/content
5. Images
6. Author and publication information
7. Sidebar
8. Related articles
9. Footer

(b) Mobile-First CSS Layout

A mobile-first design starts by designing for the smallest screen and then adds styles for larger screens.

HTML:

```
<div class="blog">
  <header>
    <h1>My Tech Blog</h1>
  </header>

  <main>
    <article>
      <h2>Introduction to Web Development</h2>
      <p>
        Web development involves creating websites
        using HTML, CSS and JavaScript.
      </p>
    </article>

    <aside>
      <h3>Related Posts</h3>
      <p>Learn CSS</p>
      <p>Learn JavaScript</p>
    </aside>
  </main>

  <footer>
    Copyright 2026
  </footer>
</div>
```

CSS:

```
body {
  margin: 0;
  font-family: Arial, sans-serif;
  line-height: 1.6;
}
```

```
main {
  display: flex;
```



```
    flex-direction: column;
    padding: 20px;
}
```

```
article,
aside {
    width: 100%;
}
```

```
img {
    max-width: 100%;
    height: auto;
}
```

The default layout is designed for mobile devices.

(c) Scaling for Larger Screens

Media queries can be used to create a larger-screen layout:

```
@media (min-width: 768px) {
    main {
        flex-direction: row;
        gap: 30px;
        max-width: 1100px;
        margin: auto;
    }
}
```

```
article {
    width: 70%;
}
```

```
aside {
    width: 30%;
}
}
```

On mobile devices, the article and sidebar appear vertically. On larger screens, they appear side by side.

(d) Accessibility and Readability

The blog should use readable font sizes, proper heading structures, sufficient contrast, and adequate spacing. Images should contain meaningful alt text. Links and buttons should be keyboard accessible. Paragraphs should not be too wide, and the layout should avoid excessive text or unnecessary animations.

Question 5: JavaScript Event Handling for Dynamic Menu – 4 Marks

(a) Appropriate JavaScript Events

The following events can be used for menu interaction:

- **click** – opens or closes the menu.
- **mouseover** – responds when the pointer moves over a menu item.
- **mouseout** – responds when the pointer leaves a menu item.
- **keydown** – supports keyboard interaction.
- **focus** – handles keyboard or accessibility focus.

For a mobile hamburger menu, the **click event** is the most appropriate.

(b) Event Handling Logic

The basic logic is:

1. Select the menu button.
2. Select the navigation menu.
3. Add a click event listener.
4. When the button is clicked, add or remove an active class.
5. CSS displays or hides the menu based on the class.

(c) Implementation

HTML:

```
<button id="menuBtn">☰</button>
```

```
<nav id="menu">
```

```
<a href="#">Home</a>
```

```
<a href="#">Products</a>
```

```
<a href="#">About</a>
```

```
<a href="#">Contact</a>
```

```
</nav>
```

CSS:

```
#menu {  
  display: none;  
}
```

```
#menu.active {  
  display: flex;  
  flex-direction: column;  
  gap: 10px;  
}
```

JavaScript:

```
const menuBtn = document.getElementById("menuBtn");  
const menu = document.getElementById("menu");
```

```
menuBtn.addEventListener("click", function() {  
  menu.classList.toggle("active");  
});
```

When the user clicks the hamburger button, the active class is added or removed. Therefore, the menu opens and closes dynamically without refreshing the webpage.

(d) Performance and Improvements

Event handling should be simple and efficient. Unnecessary event listeners should be avoided. For menus with many items, **event delegation** can reduce the number of event listeners. Accessibility can be improved by supporting keyboard navigation and using aria-expanded to indicate whether the menu is open.

Overall, JavaScript event handling makes the menu interactive while CSS controls its appearance and layout.